

DRAFT

DALOC

A Cross-Chain Prime Brokerage Protocol
for Collateralized Credit

Bartosz Nowak

bartosz@herodotus.dev

Marcello Bardus

marcello@herodotus.dev

Jamil Bouhdada

jamil@herodotus.dev

June 2026

Important Notice. This document describes the technical architecture of a protocol under development. It does not constitute an offer, solicitation, or recommendation to purchase any token, security, financial instrument, or investment product. The system described may never be deployed, or may differ materially from this description. Nothing herein constitutes financial, investment, or legal advice. References to third-party protocols are for technical context only and do not imply endorsement or partnership. Readers should consult independent professional advisors before taking any action based on this document.

Abstract

DALOC is a cross-chain credit coordination and settlement protocol for prime brokerage and structured lending. A single collateral deposit provides unified margin, leverage, and risk management across multiple chains and venues without bridging the underlying asset. Collateral stays on its home chain; yield-bearing tokens continue accruing native yield. Coordination relies on zero-knowledge storage proofs rather than trusted relayers, while token movement uses existing bridge infrastructure.

Three products share one protocol, built around independently operated markets creatable permissionlessly from governance-registered program hashes. Borrow originates loans with configurable rate models (fixed or variable) against collateral on any supported chain. Prime Trade opens a credit line from collateral to a perpetuums venue, where borrowed funds are restricted to trading by market-defined margin agents. Earn gives liquidity providers yield from a portfolio of loans: capital is deployed by an underwriter whose strategy is bounded by a committed policy hash enforced at proof time.

A common order book structure matches borrower intents against underwriter terms within each market; criteria-based filtering lets regulated and open participants share one liquidity surface without permissioned forks. Capital deploys directly from underwriter balances when available; liquidity forwarding extends reach through established lending venues, reserving only hedging capital to cover funding mismatch. LPs keep funds in their own wallets and grant allowances; capital moves only at proof-gated deployment into a specific position. Markets are defined by immutable registered zero-knowledge program hashes and parameters; risk terms are per-loan, dynamically simulated, and zero-knowledge provable. Settlement is event-driven; position terms remain off-chain commitments with only hashes and counters public. Loan positions are tokenized as transferable debt instruments with provably accruing interest.

Contents

1	Introduction	5
2	Products	6
2.1	Borrow	6
2.2	Prime Trade	6
2.3	Earn	6
3	Protocol Architecture	7
3.1	Order Book	7
3.2	Criteria-Based Matching	7
3.3	Markets	8
3.4	Hook Composition	9
3.5	Programmable Credit	9
3.6	Operators and Underwriters	10
4	Capital Management	11
4.1	Direct-Funded Path	11
4.2	Liquidity-Forwarding Path	11
4.3	Capital Efficiency	12
5	Loan Lifecycle	13
5.1	Origination	13
5.2	Refinancing	14
5.3	Close and Liquidation	14
6	The Underwriter	16
6.1	Operator and Policy	17
6.2	Compliance Enforcement	18
6.3	Capital Provision and Shares	18
6.4	First-Loss Buffer	18
6.5	LP Exit	19

7	Risk Quantification	20
7.1	Risk Objective and Loss Definition	20
7.2	Dynamic Underwriting	21
7.3	Scenario Generation	22
7.4	Provable Risk Engine	22
7.5	Funding and Execution Reserves	23
7.6	Pricing	23
7.7	Portfolio Aggregation and Concentration Risk	23
7.8	Operational Risk and Model Governance	24
8	Cross-Chain Exposure	24
8.1	Authorization Flow	24
8.2	Deterministic Addressing	25
8.3	Hyperliquid Margin Agent	25
8.4	Effective Leverage	25
9	Settlement and Verification	26
9.1	Event-Driven Settlement	26
9.2	Settlement Programs	26
9.3	Commitment Registry	27
9.4	Market and Loan Contracts	27
9.5	Oracle Freedom and Permissionless Markets	27
9.6	System Invariants	28
10	Position Tokens	28
10.1	Debt Tokens	28
11	Governance and Trust Assumptions	30
11.1	Protocol Administration	30
11.2	Liveness Dependencies	31
12	Discussion	32
12.1	Risks and Limitations	32
12.2	Future Capabilities	33

12.3 Conclusion	34
---------------------------	----

DRAFT

1 Introduction

A holder of yield-bearing collateral on one chain who wants leveraged exposure on another must unstake, bridge, and re-deposit, giving up yield and taking on counterparty risk at each step. On-chain lending exceeds \$28B [9], yet the dominant pool-based designs serve only variable-rate, open-term use cases on a single chain.

Existing protocols address important fragments of this problem. Aave provides large-scale pooled lending with governance-managed risk parameters, isolation modes, e-mode, and a hub-and-spoke architecture in Aave V4 [1]. Euler V2 provides a modular lending framework through isolated vaults and the Ethereum Vault Connector, letting builders compose customized credit markets [10]. Morpho Blue separates lending infrastructure from risk curation through isolated markets, while Morpho’s fixed-maturity work moves closer to intent-based, fixed-term credit backed by tokenized debt [11, 19]. Spark focuses on conservative stablecoin lending and protocol-directed capital allocation through SparkLend and related liquidity infrastructure [25]. Vault-based systems further improve allocation by curating exposure across markets [29].

These systems improve capital efficiency, market customization, and risk curation. None simultaneously coordinates structured, term-bounded credit (fixed and variable rate models) across chains, keeps collateral on its home chain while proving its state without trusted relayers, sources capital from external lending venues, converts variable external-venue funding into borrower-facing fixed or index-linked terms, routes borrowed capital into constrained execution environments, and accommodates institutional criteria without fragmenting liquidity. Public materials and roadmaps for the major lending protocols show overlap with individual components, especially modular markets, curated vaults, fixed-term loans, and cross-chain architecture, but not this full combination inside one settlement and verification layer [1, 10, 11, 19, 25].

DALOC unifies these capabilities in a single credit coordination and settlement layer. Fixed- and variable-rate term loans, leveraged perpetual trading, and passive yield share a common matching and settlement infrastructure; borrowers, traders, and liquidity providers interact through the same intent structure without bridging collateral or fragmenting liquidity. Capital deploys from underwriter balances when available, or via liquidity forwarding through established lending venues; criteria-based matching accommodates regulated participants alongside open ones. Programmable zero-knowledge markets, a provable risk engine, and permissionless proof-gated execution complete the architecture; independently operated markets isolate liveness and trust assumptions. The decentralization properties of this architecture, including correctness, completeness, and exit guarantees, are analyzed in the companion paper [20]. The architecture carries inherent risks (smart-contract vulnerabilities, oracle dependencies, bridge liveness, and proof-system availability) discussed in Sections 11 and 12.

2 Products

All three products share a single protocol layer (the underwriter, per-market order book, settlement pipeline, and risk engine), extensible to new products without modifying core contracts.

2.1 Borrow

A holder of any supported token borrows against it with configurable rate models (fixed-rate or variable-rate referencing an external index) for a defined term. Collateral stays on its home chain; yield-bearing collateral (e.g. wstETH) continues accruing native yield throughout the loan. Loan-to-value ratio, rate, and term are determined per loan through operator competition (Section 3). Origination, interest accrual, and liquidation are each governed by zero-knowledge programs, ensuring that no single trusted party controls the loan's lifecycle.

2.2 Prime Trade

A borrower draws a credit line in the market's supported principal asset and routes it to a per-loan margin agent on HyperEVM, gaining leveraged perpetual exposure without selling the underlying collateral (Section 8). In the initial markets, USDC is the canonical principal asset because of its deep DeFi liquidity. USDC is therefore the reference implementation, not a permanent protocol constraint: other stablecoins may be supported by separate markets if the relevant liquidity, risk, and compliance requirements are satisfied.

The margin agent restricts capital to approved operations (trading only, no withdrawals), so the lender's risk depends on programmatic constraints, not borrower discretion. Venue leverage multiplies the borrowing LTV, controlling large notional with modest collateral (Section 8).

2.3 Earn

Liquidity providers grant an allowance to an underwriter and receive ERC-20 shares representing their proportional claim on deployed capital and returns. Proof-gated settlement deploys underwriter capital into loans originated through the protocol's order book, either funding loan principal directly or providing hedging buffers when loans are sourced through external venues (Section 4). The underwriter's strategy is bounded by a committed policy hash; compliance is enforced at proof time (Section 6). Underwriter policy selects the rate model: under fixed-rate terms the coupon is locked at origination; under variable-rate terms an external index is passed through to borrowers. On direct-funded loans, yield derives from the full coupon; on liquidity-forwarding loans, from the hedging-buffer surplus. LP funds leave their wallets only at the instant of proof-gated deployment into a specific position.

3 Protocol Architecture

3.1 Order Book

All loan activity flows through an order book maintained off-chain by each market's operator (the party that performs matching and produces zero-knowledge proofs; independent keepers submit them on-chain for execution [20]). Borrower requests and underwriter terms use the same intent structure: collateral, principal, rate, term, and execution path (the full quote specification is given in Section 7). Each intent includes a nonce; the creation proof's on-chain execution writes a nullifier derived from the intent nonce, so a borrower can safely broadcast the same intent to multiple operators: only the first successful proof executes. The operator selects among criteria-compatible quotes; the creation proof verifies that signed intents are valid, criteria are satisfied, and proposed terms fall within the underwriter's committed policy. Routing discretion—which quotes to match and to which underwriter—is bounded by competition, not by the proof [20]. Quote terms are visible only to the operator, the underwriter, and the borrower.

Participation requires no upfront capital lock. A borrower signs an off-chain intent and grants a token allowance to the hub, authorizing a future transfer but moving nothing. Capital is transferred only when a settlement proof executes the match on-chain; until then, funds remain in the participant's wallet.

3.2 Criteria-Based Matching

Beyond price compatibility, a match must satisfy *criteria*: typed attributes attached to an intent (counterparty allowlists, jurisdictional restrictions, KYC-tier requirements, or sanctions-screen attestations). The operator treats a pair as compatible only if each side's criteria are satisfied by the other's attributes. Unrestricted participants cannot match against restricted entries whose criteria they do not satisfy; restricted participants match only against compatible counterparties. There is no separate permissioned market; entries in one order book vary only in how restrictive they are.

This avoids the fragmentation of permissioned forks: one liquidity surface, one risk engine, one settlement pipeline, one audit perimeter. Adding a new jurisdiction means adding a new attestation issuer, not deploying new contracts.

Criteria can be committed-only (stored as a hash, evaluated inside the proof, revealing only the boolean outcome). Counterparty attributes are typically committed-only, preserving privacy while still enforcing compatibility. The default is open: an unrestricted intent imposes no criteria on its counterparty. Criteria proofs are a protocol-level matching primitive; they do not constitute or replace compliance with applicable legal or regulatory obligations, which remain the responsibility of each participant.

3.3 Markets

Criteria restrict *who* can match; markets define *what* the matched position looks like. DALOC is organized around isolated lending markets. Each market is uniquely identified by a deterministic hash of its parameters: the collateral chain and asset, principal chain and asset, and four zero-knowledge program hashes that govern loan behavior, including accrual, liquidation, collateral policy, and oracle-dependent state transitions. All parameters are fixed at creation; a different accrual program, liquidation rule, collateral policy, or oracle policy constitutes a different market. Within a market's accrual program, rate model selection (fixed or variable) is a per-position decision set by the underwriter's committed policy (Section 6).

Oracle selection follows the same immutability rule. In protocols such as Morpho Blue, the oracle is an explicit market parameter: each market points to a specific oracle contract, fixed at creation [11]. In DALOC, oracle logic is embedded in the market's zero-knowledge programs instead. A market program specifies which on-chain sources may be read, how prices or rates are transformed, and which freshness constraints must hold before a state transition is accepted. The protocol verifies the program output, not a protocol-wide oracle choice. Changing the oracle source, aggregation rule, freshness bound, or liquidation-price logic changes the program hash and therefore defines a new market.

Markets are created through the *hub* contract (DalocHub), deployed at identical addresses on every supported chain (Section 8). Creating a market from registered program hashes is permissionless; deploying a wholly new market *type* (novel risk model with new proof programs) requires the program hashes to be registered through the hub's governance process. Each chain's hub instance is the sole token rail on that chain: all user-facing ERC-20 transfers route through it, creating a single per-chain authorization point for capital flows, intent cancellations, and global pause. The hub maintains zero persistent balance; protocol fees are deducted and routed to a fee-accumulator contract atomically at settlement time.

Different markets can encode different collateral strategies: a *classical* market locks collateral as a liquidation hedge, while a more aggressive market can permit managed rehypothecation. The choice is a market-definition decision, not a protocol-level toggle.

Each market deploys issuer contracts on both the collateral and principal chains; each funded loan creates per-loan contracts at deterministic CREATE2 addresses. Per-loan contracts are isolated (one per loan, not shared across borrowers), so one position's risk cannot cascade to another.

Isolation extends beyond individual positions to entire markets. Each market operates as an independent state machine: it maintains its own state and shares no global state with other markets. A market can access only its own bal-

ances; one market's bug or misconfiguration cannot reach into another's funds. Contract-level isolation is structural, but economic exposure is shared within an underwriter: bad debt from any position hits the common first-loss buffer, and if the buffer is exhausted, LP share price across the underwriter's entire portfolio is impaired (Section 6). Multiple markets for the same collateral/principal pair may coexist under different risk parameters, oracle policies, rule sets, or execution operators; participants evaluate each market's operator, parameters, and track record before choosing where to transact.

3.4 Hook Composition

Lending operations are expressed as atomic hook chains attached to settlement events. Each transfer in the settlement tree carries optional *pre-hooks* and *post-hooks*: pre-hooks execute before the transfer (e.g. deploy a CDP), the transfer moves collateral or principal, and post-hooks execute after (e.g. supply to an external venue, borrow, bridge). All steps on a single chain must succeed or the entire on-chain operation reverts; cross-chain steps (bridge hooks) are committed atomically but execute asynchronously on the destination chain, stalling rather than reverting if the bridge is delayed (Section 8).

Operators assemble hook chains and commit them into the settlement tree before on-chain execution. The settlement proof verifies that every hook is authorized by the market's ZK programs and that the resulting state satisfies the market's invariants. Borrowers sign high-level intents; operators compete to attach the cheapest valid execution plan. Intent cancellation is a direct user action requiring only the user's signature, with no proof and no operator: it takes effect immediately on-chain, and subsequent creation proofs verify cancellation status via storage proof. Margin adjustment (adding collateral) and direct repayment are likewise direct user actions [20].

3.5 Programmable Credit

Permissionless markets, hook composition, and constrained execution together form a programmable credit layer. A market creator writes ZK programs governing accrual, liquidation, and collateral policy; operators compete to assemble the cheapest valid execution plan from hooks; borrowers express only high-level intents.

Constrained execution environments extend this programmability to the destination chain. A margin agent on HyperEVM restricts borrowed capital to approved operations (perpetual trading only, no withdrawals), but the constraint boundary itself is programmable: HyperEVM contracts interact with HyperCore's perpetual engine via precompiles, enabling on-chain strategies within the lender-defined safety envelope:

- **Delta-neutral yield.** A borrower pledges stETH, borrows USDC, and the margin agent automatically opens a short ETH perpetual matching the col-

lateral's notional. The position is market-neutral; net yield equals the staking rate minus funding cost minus the DALOC fixed borrowing rate. The constraint permits only a single, fully hedged position with no directional bets.

- **Automated stop-loss.** The margin agent monitors unrealized PnL via pre-compile reads and auto-closes the position if losses exceed a configurable threshold, bounding lender exposure during inactive periods.
- **Portfolio rebalancing.** The margin agent maintains multiple perpetual positions at target allocations, rebalancing when drift exceeds a bound and keeping margin utilization within the market's declared limits.

Because the lender's risk depends on what capital *can* do, not on borrower intent, tighter constraints enable more favorable terms and may, in future, support under-collateralized credit lines.

3.6 Operators and Underwriters

An *operator* matches borrower intents against underwriter capital and produces zero-knowledge proofs for each settlement action. Independent *keepers* submit these proofs on-chain for execution fees; independent *liquidators* enforce against positions that breach committed parameters for bonuses. The operator never holds tokens; its scope is matching and proof generation. Revenue is a matching fee on successful originations. Two behaviors remain unconstrained by proofs: liveness (whether to produce proofs at all) and ordering (which intents to match, and to which underwriter); both are bounded by competition and deadlines [20].

An *underwriter* is an on-chain entity that pools LP capital, underwrites position risk, and stands as economic counterparty to borrowers (Section 6). Each underwriter is scoped to one operator's registry, establishing clean separation: losses in one scope cannot reach another. Each underwriter commits a policy hash encoding its operating constraints: risk parameters, rate ranges, maturity limits, maximum exposure, bonus ranges, and sweep frequency. The operator routes intents to underwriters; the creation proof verifies that proposed terms fall within the underwriter's committed policy.

Multiple underwriter variants serve different strategies. An *automated underwriter* originates positions if and only if they satisfy the committed policy hash: all decisions are deterministic and provable. A *curator-managed underwriter* adds active management under an enforced mandate: a curator signs per-position authorizations within bounds checked at proof time [20]. Other structures (direct lenders, external venue wrappers) plug into the same interface; the operator treats all variants uniformly.

Routing and capital sourcing are operator-level and underwriter-level concerns respectively, making the protocol extensible to new venues without core con-

tract changes. DALOC provides the settlement rails; existing venues provide the liquidity.

4 Capital Management

The underwriter is the canonical capital source. When available LP capital exceeds the requested principal, a creation proof executes a direct-funded transfer from the underwriter's allowance to the borrower; the underwriter receives debt tokens as the creditor of record. The operator produces the proof but never custodies tokens. When demand exceeds available capital, the creation proof's hook chain sources principal from external lending venues instead, reserving underwriter capital only for hedging instruments that cover funding mismatch (rate buffers for fixed-rate loans; index-linked reserves for variable-rate loans, per underwriter policy). The underwriter's committed policy (Section 6) governs the allocation between paths.

4.1 Direct-Funded Path

The underwriter deploys principal directly to the borrower (or to the margin agent for Prime Trade). The CDP holds collateral as a liquidation hedge; no external venue is involved. Capital is committed at 1:1 with no variable-rate mismatch or hedging buffers, and the underwriter captures the full coupon (fixed or variable, depending on the rate model). This path is preferred whenever available capital permits.

4.2 Liquidity-Forwarding Path

When available LP capital is insufficient (because existing deployments have consumed it, or because the loan size exceeds available balance), the creation proof's hook chain sources principal from established lending venues via the CDP rather than from the underwriter's balance sheet directly; the operator produces the proof but never custodies tokens. The CDP deposits pledged collateral, or an approved intermediate asset after a collateral swap, into an admissible external venue, borrows the market's supported principal asset against it, and routes the borrowed asset to the borrower or margin agent. For the initial USDC-based markets, admissible venues may include Aave [1], Morpho [11], and Spark [25]. The underwriter commits only a fraction of principal as hedging reserves (rate buffer plus collateral conversion reserve), achieving high capital leverage. Pledged collateral simultaneously secures the DALOC loan and the external venue position.

External venues are not selected arbitrarily by operators at execution time. Each market defines its admissible venue set through its zero-knowledge programs and authorized hook adapters. A venue is admissible only if its collateral support, principal asset, oracle behavior, liquidation mechanics, rate model, and

on-chain state can be verified by the market’s programs and incorporated into the risk engine. Changing the venue set, adapter, or verification logic therefore defines a different market.

When underwriter capital is used for hedging reserves, the underwriter’s committed policy may impose stricter constraints than the market itself: allowed venues, maximum exposure per venue, maximum utilization or borrow-rate assumptions, minimum liquidity depth, liquidation-latency bounds, and adapter-level risk limits. The creation proof attests that liquidity-forwarding activity stays within this policy; a missing or invalid proof prevents origination.

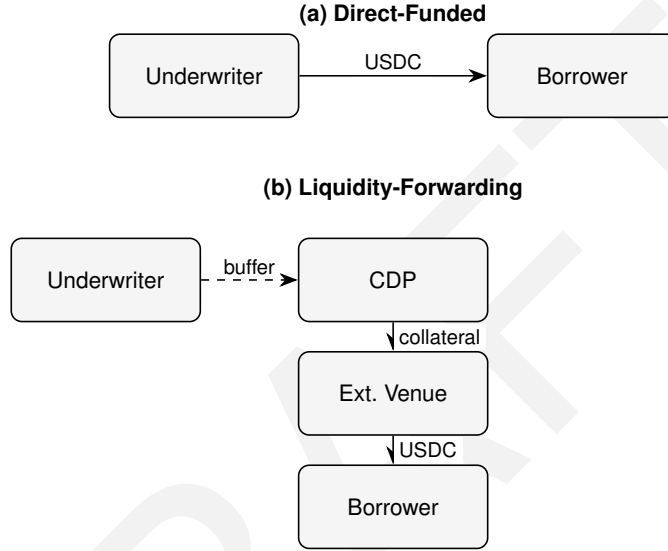


Figure 1: Capital paths. **(a)** Direct-funded: the underwriter deploys principal directly and captures the full coupon. **(b)** Liquidity-forwarding: the CDP borrows from the external venue; the underwriter commits only hedging capital (dashed), achieving $\sim 20:1$ leverage.

4.3 Capital Efficiency

Both buffers on the liquidity-forwarding path are sized at quote time by the provable risk engine (Section 7): the rate buffer from tail simulations of variable-rate paths, the collateral conversion reserve from simulated round-trip execution costs. For a liquidity-forwarding loan, the underwriter’s capital leverage is

$$\kappa = \frac{P}{B_{\text{rate}} + B_{\text{conv}}}, \quad (4.1)$$

where P is the loan principal and $B_{\text{rate}} + B_{\text{conv}}$ is the total hedging capital committed. On the direct-funded path the underwriter commits the full principal ($\kappa = 1$ by definition); on the liquidity-forwarding path, typical combined buffers of $\sim 5\%$ of principal achieve $\kappa \approx 20$. The blend between paths depends on available LP capital and market demand.

At close, buffer surplus remains as LP yield; buffer shortfall reduces underwriter NAV.

5 Loan Lifecycle

5.1 Origination

A borrower posts a request, the operator matches it against underwriter terms, and the accepted match is committed into a settlement proof and executed atomically on-chain. Which capital path applies (Section 4) determines the pipeline complexity. Three capital pipelines exist, each assembled as an atomic hook chain, with an optional Prime Trade routing step on the final transfer:

Direct-funded. The underwriter deploys USDC principal directly to the borrower (or to the margin agent for Prime Trade positions); the CDP holds collateral as a liquidation hedge without supplying it to an external venue, and no hedging buffers are reserved.

Liquidity-forwarding. The hook chain deposits collateral into the collateral manager, supplies it to the external venue, borrows USDC against it, and transfers the USDC to the borrower. For fixed-rate borrower loans, a rate buffer is escrowed to hedge the mismatch between the borrower's fixed rate and the venue's variable rate; variable-rate loans reference the committed index and size reserves according to the underwriter's policy.

Liquidity-forwarding with collateral swap. For collateral not directly supported by the selected external venue (e.g. LDO or UNI), the market may authorize a swap hook that converts the pledged collateral into a venue-listed intermediate asset (e.g. LDO → WETH) through an approved AMM, RFQ venue, or aggregator before the hook chain proceeds as above. This path is available only for collateral assets, intermediate assets, swap adapters, and execution routes admitted by the market's zero-knowledge programs and, where applicable, by the underwriter's committed policy.

The swap hook is executed with explicit price-protection constraints committed in the settlement leaf (minimum output, deadline, maximum slippage). If these constraints are not satisfied at execution time, the hook reverts and the origination does not settle. Both a rate buffer and a collateral conversion reserve are escrowed; the conversion reserve is sized by the risk engine to cover stressed round-trip execution costs. In canonical DALOC markets, assets or routes with insufficient oracle quality, shallow executable liquidity, or unavailable liquidation paths are rejected rather than routed through the swap path.

Hyperliquid margin. For Prime Trade positions, the hook chain follows either pipeline above but diverts at the final step: instead of transferring USDC to the borrower, a bridge hook routes it to a per-loan margin agent on HyperEVM, which deposits it as margin (Section 8). Collateral never leaves Ethereum.

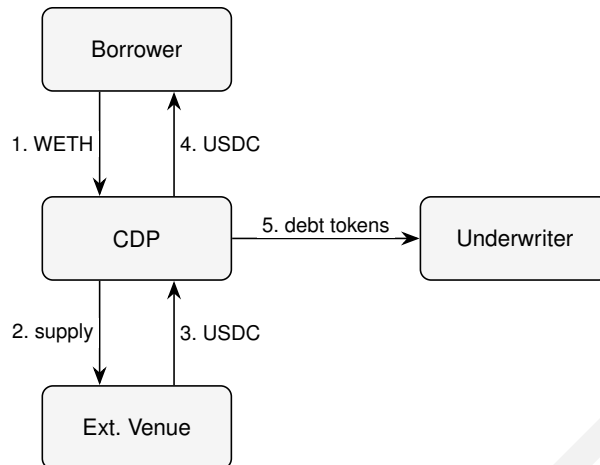


Figure 2: Origination, liquidity-forwarding pipeline. Steps 1–4 execute as a single atomic hook chain. Debt tokens (step 5) are minted to the underwriter; the rate buffer is escrowed. Direct-funded origination omits the external venue and transfers principal directly.

5.2 Refinancing

Not every loan runs to its original maturity. A borrower refinances by posting a single refinancing intent; the matching engine treats it identically to a new origination request. On match, the existing loan closes and a new one originates atomically within the same settlement proof. When the same underwriter is creditor on both sides of a direct-funded loan, refinancing is a pure accounting event: no capital moves, only terms change. When the creditor differs or the loan involves an external venue, repayment and new funding settle atomically. For Prime Trade positions, refinancing applies to the credit terms on Ethereum; margin capital on HyperEVM remains in place unless the position is fully closed and re-originated.

5.3 Close and Liquidation

Close is permitted at or after maturity (mid-term exit is available only through refinancing, as described above); either counterparty can produce the close proof independently [20]. On voluntary close, the borrower deposits the principal asset into the loan’s debt contract. After interest accrual is verified (Section 10), the creditor (underwriter or direct lender) redeems its position and a close hook chain repays the variable debt on the external venue, withdraws the collateral, and returns it to the borrower (with a reverse swap for collateral-swap loans). Hedging surplus stays with the capital provider as yield. Direct-funded loans skip the external unwind: the borrower repays, and collateral is released directly. For Prime Trade positions, the close hook chain first recalls capital from the destination venue via the bridge before repaying the source-chain debt.

If the borrower has not repaid at maturity, the creditor produces a close proof that establishes the owed amount; the contract writes an enforcement authorization flag, enabling any liquidator to execute under the same flag-gated model described below [20].

The external unwind is subject to venue-level liquidity and liveness constraints. If the venue cannot provide the required borrow liquidity at origination, or if utilization, borrow rate, pause status, or available liquidity violates the bounds committed in the quote, the origination hook reverts and the loan does not settle. For active liquidity-forwarding loans, a stressed venue can increase variable funding costs, delay collateral withdrawal, or force execution through a slower liquidation route. These risks are modeled by the risk engine through rate buffers, collateral-conversion reserves, venue exposure limits, and liquidation-latency assumptions.

If a close or liquidation cannot fully unwind the external venue position because the venue has insufficient available liquidity, withdrawals are paused, or the adapter reports a failed unwind, the loan enters a pending-unwind state. Debt repayment and recovered assets are ring-fenced, borrower withdrawals are delayed until the external position is unwound or auctioned, and any excess funding or execution cost is charged first against loan-specific reserves. Residual shortfall follows the loss waterfall (Section 6). DALOC cannot remove external venue liquidity risk; it makes the risk explicit, priced, reserved, and isolated per loan.

Liquidation triggers when the health factor drops below 1:

$$\text{HF} = \frac{C_{\text{val}} \theta}{D} < 1, \quad (5.1)$$

where C_{val} is the collateral's current market value, θ the loan's liquidation threshold (set at origination, strictly above the origination LTV), and D the outstanding debt including accrued interest. Because D grows with interest accrual, the risk engine constrains (θ, λ, r, T) so that the health factor remains above 1 at maturity under constant collateral value. For fixed-rate positions this is a deterministic check; for variable-rate positions the constraint is probabilistic, sized so that HF remains above 1 under the stressed rate scenarios produced by the precomputation kernel (Section 7.4).

Liquidation uses a flag-gated, permissionless execution model.

Detection. A health-check sweep (produced by the operator) or a per-position proof (producible by either counterparty) computes current true debt internally from the position's rate model and elapsed time since last on-chain accrual (read via storage proof); for variable-rate positions the circuit also reads the external rate index via storage proof. The health assessment therefore does not depend on accrual proofs having been submitted recently. The circuit reads collateral price and liquidation threshold from on-chain state via storage proofs, asserting $\text{HF} < 1$. On verification, the contract writes an *authorization flag* for each

breached position. Detection is identity-agnostic and capital-free: the prover reveals nothing beyond the breach. Per-position enforcement is not a fallback; it is a guaranteed, immediate capability of either counterparty, independent of operator liveness [20].

Recovery window. Between detection and execution, the borrower may add collateral or partially repay debt to restore $HF > 1$. A subsequent sweep or per-position health attestation proof clears the flag if the position has genuinely recovered. The staleness bound on oracle readings prevents gaming via stale favorable prices. Between flag-setting and flag-clearing, a liquidator may execute against a position whose current health is above threshold; the borrower is compensated at execution-time oracle price (receiving residual collateral after debt is paid) but loses the bonus fraction. Timely health attestation by the borrower is the mitigation [20].

Execution. Once an authorization flag exists, any liquidator can execute without permission from operator, underwriter, or governance. Liquidation is atomic at the contract level: the first valid executor claims the bonus. The liquidator repays outstanding debt R via the debt token contract and seizes collateral worth $R \cdot (1 + \beta)$ from the CDP, where β is the liquidation bonus fixed at origination. The contract reads the oracle price at execution time, ensuring the price reflects conditions at the moment of liquidation. For liquidity-forwarding loans, the seizure hook chain unwinds the external venue position; if the venue cannot fully unwind (paused withdrawals, insufficient liquidity), the loan enters the pending-unwind state described above. Residual collateral beyond the liquidator's claim is returned to the borrower. Positions where collateral value falls below outstanding debt become uneconomic to liquidate; these bad-debt losses are absorbed by the loss waterfall (Section 6).

Because enforcement is authorization-gated (the flag must exist) rather than proof-gated at execution time, the contract reads the oracle directly during the liquidation transaction. Flash loans eliminate the capital requirement for liquidators [21, 22]: borrow the debt, seize collateral at a discount, sell, repay, keep the difference.

Because each loan is isolated, bad debt cannot cascade across positions. Shortfalls reduce underwriter NAV through impaired debt-token value (Section 6).

6 The Underwriter

An underwriter aggregates LP capital, underwrites position risk, and stands as economic counterparty to borrowers. The operator matches intents, assembles hook chains, and produces settlement proofs within each underwriter's committed policy; lifecycle proofs (accrual, close, enforcement) are independently producible by either counterparty [20]. All actions are bounded by a committed policy hash; compliance is enforced at proof time. The underwriter is the canonical lending book: when it has sufficient available capital, origination deploys

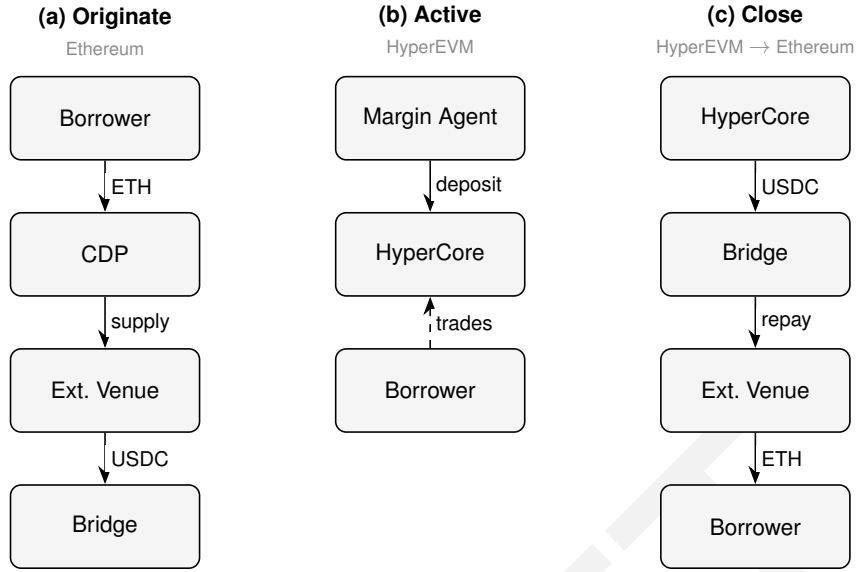


Figure 3: Prime Trade lifecycle (liquidity-forwarding variant shown; direct-funded Prime Trade skips the external venue steps). **(a)** Borrower pledges ETH; CDP supplies to the external venue; USDC bridged to destination. **(b)** Margin agent deposits into HyperCore; borrower trades (dashed). **(c)** Capital recalled via bridge; venue repaid; collateral released.

it directly at the full coupon (Section 4); when demand exceeds capacity, origination uses the liquidity-forwarding hook chain and the underwriter commits only hedging reserves.

6.1 Operator and Policy

The operator submits quotes to the order book (Section 3), competing on terms; on match, keepsers execute settlement proofs that deploy underwriter capital within committed policy bounds. The underwriter’s strategy is bounded by a **committed policy**: a hash registered on-chain at deployment. Each policy specifies:

- **Allowed collateral set:** asset addresses the underwriter may accept as loan collateral.
- **Per-collateral LTV ceiling:** maximum loan-to-value ratio the underwriter may offer on each collateral type.
- **Per-tenor rate floor:** minimum annualized rate the underwriter must charge.
- **Liquidation bonus ranges:** acceptable bonus percentages committed at origination.
- **Portfolio concentration caps:** maximum exposure per borrower, per collateral type, and per market as a fraction of underwriter NAV.
- **Sweep frequency:** maximum interval between health-check sweeps.

- **Minimum buffer ratio:** minimum first-loss buffer maintained at all times relative to total exposure.

A policy constrains what the operator *can* do, not what they *must* do; within its bounds, the operator retains full discretion on timing, counterparty selection, and position sizing.

6.2 Compliance Enforcement

Every origination proof verifies that the proposed position satisfies the underwriter's committed policy hash. The circuit checks that funded loans, hedging reserves, external venue choices, concentration exposure, rate floors, LTV ceilings, and buffer ratios remain within the policy's bounds. An action that violates the policy cannot produce a valid proof.

The detailed proof format, public inputs, and fact verification flow are part of the settlement-program framework described in Section 9. The decentralization paper [20] specifies how compliance enforcement operates without reliance on the operator.

6.3 Capital Provision and Shares

LPs keep funds in their own wallets and grant the underwriter an on-chain allowance. In return, they receive transferable ERC-20 shares representing proportional claims on deployed capital and returns. Funds leave LP wallets only at the instant of proof-gated deployment into a specific position.

Share price reflects the underwriter's net asset value as recorded on-chain:

$$\text{NAV} = \sum_j v_j \cdot n_j + B_{\text{buffer}} + I_{\text{proven}}, \quad (6.1)$$

where $v_j \cdot n_j$ is the claim value (Equation 10.1) times quantity for each held debt token j , B_{buffer} is the first-loss buffer balance, and I_{proven} is interest verified by accrual proofs but not yet applied to the corresponding debt tokens' on-chain claim values v_j (the increment between fact registration and debt-token state update). Because accrual proofs update balances discretely, on-chain NAV conservatively lags mathematically accrued interest between proof submissions; accrual proof producers are compensated with a deferred fraction of accrued interest, incentivizing timely accounting. Share price is NAV per outstanding supply, with a virtual offset against inflation attacks. On direct-funded loans, the underwriter earns the full coupon (fixed or variable, depending on the rate model); on liquidity-forwarding loans, the underwriter earns buffer surplus. Impairments reduce NAV directly via share-price adjustment.

6.4 First-Loss Buffer

Protection comes from a first-loss buffer funded primarily by per-origination fees. When a position produces bad debt, the buffer absorbs the shortfall before

share price moves. The creation proof enforces a minimum buffer-to-exposure ratio, blocking new originations when the buffer thins. The loss hierarchy:

1. borrower repayments and collateral value;
2. loan-specific funding and execution reserves;
3. permissionless liquidation;
4. underwriter first-loss buffer;
5. residual LP impairment.

One limitation: the on-chain buffer ratio reflects only realized losses (positions that have been enforced and closed at a loss). Positions in bad-debt territory but not yet enforced still appear as healthy exposure, potentially overstating buffer adequacy. Timely enforcement by counterparties mitigates this gap [20].

Because the buffer is funded primarily by origination fees, it faces a procyclical gap: during a downturn, new origination drops precisely when existing positions may be deteriorating, starving the buffer of inflows. To prevent an irrecoverable state where buffer depletion permanently blocks origination, the underwriter contract accepts direct capital injections into the buffer from any source. This injection is voluntary and does not dilute LP shares; it restores origination capacity.

6.5 LP Exit

LP exit follows the allowance-and-shares model: revoke the allowance and further deployments from that wallet stop immediately. Shares on already-deployed capital remain outstanding until underlying positions close and capital returns, or the LP sells shares on secondary markets for immediate liquidity. The critical stability property: exit constrains future origination capacity, not existing healthy positions, so withdrawal pressure cannot trigger asset sales or position unwinds [20].

Table 1 decomposes risk by participant and origination path.

Party	Risk borne	Why	Compensation
Borrower	Collateral decline	Seeks leverage	Access to capital
Ext. venue	Smart-contract, util.	Money market	Variable rate
Rate buffer	Variable > fixed	Rate transform	Surplus
Conv. reserve	Swap cost > escrow	Execution risk	Escrow ~95%
First-loss buffer	Liquidation shortfall	Backstop capital	Origination fees
LP	Default, exhaustion	Absorbs residual	Coupon or surplus
Operator	Liveness, reputation	Proof generation	Matching fee

Table 1: Risk decomposition. Each risk channel is routed to the participant with the comparative advantage to bear it. External venue, rate buffer, and conversion reserve rows apply only to liquidity-forwarding loans; direct-funded loans skip directly from borrower to first-loss buffer and LP.

7 Risk Quantification

The underwriter’s committed policy (Section 6) sets the outer boundaries (collateral classes, LTV ceilings, rate floors, concentration caps). The risk engine sets the per-loan terms within those boundaries. This section specifies how DALOC converts risk channels (Table 1) into loan-level loss distributions, underwriting terms, reserve requirements, and buffer-adequacy checks before capital is deployed.

Each loan is treated as a collateralized credit instrument, not merely as an exposure to collateral volatility. Structural credit models show that lender risk depends on the relation between collateral value, debt, maturity, and recovery, not on volatility alone [17]. DeFi lending markets add endogenous funding and liquidity risk: interest rates, utilization, and market liquidity change with protocol conditions and can be correlated across venues during stress [13]. Recent work models decentralized lending contracts as option-like claims whose lender payoff depends on LTV, liquidation rules, transaction costs, and hedging feasibility [26]. Vault-based credit adds another layer: depositors bear losses through liquidation failure, oracle execution divergence, liquidity constraints, governance limits, and smart-contract risk [30]. For this reason, DALOC models the full economic loss distribution after reserves, recovery, and buffer commitments, rather than using volatility or liquidation probability as a standalone proxy for lender risk.

7.1 Risk Objective and Loss Definition

The risk engine constrains economic loss to the lender, the underwriter, and its first-loss buffer under both ordinary and stressed conditions. A liquidation event is not itself a default. A position may be liquidated without lender loss if the debt is repaid in full, while lender loss can arise when liquidation is delayed, market depth is insufficient, execution costs are high, or funding and settlement costs exhaust loan-specific reserves. Empirical work on DeFi liquidations shows that liquidation incentives, collateral discounts, and liquidator behavior materially affect realized outcomes [21, 22]. Recent work on liquidation dynamics further shows that transaction fees, oracle-extractable value, and market microstructure can affect whether a liquidation protects lenders or amplifies loss [24]. Underwriter-level credit analysis therefore treats liquidation mechanics as only one component of depositor loss, alongside oracle, liquidity, governance, and infrastructure risks [30].

For each loan i , let ρ_i denote a realized scenario path. A scenario path includes collateral prices, funding rates, venue liquidity, oracle updates, execution costs, bridge or settlement latency, and liquidation outcomes over the loan horizon. Let $EAD_i(\rho_i)$ be the exposure at close-out: contractual debt, accrued obligations, funding deficits, execution costs, and settlement costs under scenario ρ_i . Let $R_i^{\text{pre}}(\rho_i)$ be pre-buffer recovery: borrower repayments, collateral proceeds, and

loan-specific reserves available before any buffer payout. The gross economic loss before buffer support is

$$L_i^{\text{gross}} = \max(\text{EAD}_i(\rho_i) - R_i^{\text{pre}}(\rho_i), 0).$$

If loan i is covered by an underwriter with first-loss buffer commitment A_i , the buffer payout and residual lender loss are

$$L_i^{\text{buffer}} = \min(L_i^{\text{gross}}, A_i),$$

$$L_i^{\text{net}} = \max(L_i^{\text{gross}} - A_i, 0).$$

Thus L_i^{gross} is the loss before buffer support, L_i^{buffer} is the amount absorbed by the first-loss buffer, and L_i^{net} is the residual loss borne by LPs through share-price impairment. A loan can therefore be liquidated successfully without lender loss when recovery covers the debt; the buffer absorbs shortfall up to its commitment; residual loss, if any, falls to LPs.

The model separately reports three quantities. The liquidation probability is the probability that the loan breaches its health factor and enters liquidation. The loss probability is the probability that residual economic loss is positive, $\Pr[L_i^{\text{net}} > 0]$. Loss given default is the expected residual loss conditional on a positive loss event,

$$\text{LGD}_i = \mathbb{E}[L_i^{\text{net}} \mid L_i^{\text{net}} > 0].$$

At portfolio level, losses are aggregated under the same joint scenario, preserving dependence through common market, liquidity, oracle, funding, bridge, and operational shocks. Expected shortfall is used as the primary tail-risk measure because it captures the severity of losses beyond a quantile and supports portfolio aggregation and constrained optimization [2, 23]. For a portfolio loss random variable L , the expected shortfall at confidence level α is

$$\text{ES}_\alpha(L) = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha(L)],$$

with the confidence level, liquidation horizon, stress set, and admissibility bounds disclosed by each underwriter and bounded by the underwriter's committed policy and declared precomputation kernel hash (Section 7.4).

7.2 Dynamic Underwriting

Every quote is produced from the current state of the collateral, principal asset, funding path, liquidation venues, and existing portfolio. The quote specifies at least

$$q_i = (N_i, \lambda_i, \theta_i, r_i, T_i, B_i, \pi_i), \quad (7.1)$$

where N_i (gross principal), λ_i (origination LTV), and θ_i (liquidation threshold) define the position; r_i (rate or committed rate-model identifier) and T_i (term) define the economics; B_i (reserve vector) sizes the hedging; and π_i identifies the underwriter's committed policy hash.

Principal limits, rate, liquidation threshold, reserve sizing, and admissibility are outputs of the underwriting process, not protocol constants. Fixed contractual terms are not repriced ex post; a deterioration in risk may trigger a top-up request, refinancing, or liquidation according to the market’s pre-committed rules.

7.3 Scenario Generation

Risk is estimated under the real-world probability measure because the objective is capital adequacy, not only no-arbitrage valuation. The scenario generator reproduces volatility clustering, jumps, heavy tails, and the tendency of liquidity, funding rates, and operational latency to worsen together during stress [6, 8, 13, 15, 28]. Challenger models (alternative specifications run against the primary model to detect divergence) run in parallel; parameter uncertainty is converted into conservative add-ons.

Each simulated path replays actual loan cash-flow logic (accrual, liquidation detection, and recovery), producing a distribution of economic loss, not merely of collateral prices. Recovery is modeled as endogenous to trade size, venue depth, and liquidator economics [21, 22, 24, 27]. For liquidity-forwarding loans, reserves are sized so that under modeled stress scenarios the protocol’s liquidation completes before the venue’s instant trigger fires; tail scenarios beyond the modeled set may still produce a gap (Section 12).

7.4 Provable Risk Engine

The risk engine separates computation into two layers: a precomputation kernel and a serving path. The precomputation kernel is a hash-committed risk surface that encodes the heavy simulation output used to bound quote generation, reserve sizing, and portfolio aggregation. The serving path answers individual quote requests against this committed kernel and produces a zero-knowledge proof of correct computation.

Precomputation kernel. Simulation results are materialized into a multi-dimensional tensor indexed by market-driven and loan-level parameters (volatility regime, LTV, rate, term). Each cell stores break-even quantities, liquidation probabilities, loss quantiles, reserve requirements, and admissibility bounds at which the protocol’s expected net loss is zero or better. Only the computationally heaviest quantities reside in the kernel; all remaining quote parameters are derived in the serving path. Recomputation happens periodically from fresh market data; between updates, the same committed kernel serves all quotes for a given collateral/principal pair.

Serving path. When a borrower requests a quote, the serving layer maps the request into the kernel’s parameter space and searches for the optimal

$(\lambda_i, \theta_i, r_i, B_i)$ that satisfies the borrower’s intent while remaining break-even or better for the lender. Interpolation over the discrete grid yields a continuous surface; optimization proceeds over it without Monte Carlo at serving time, reducing latency by orders of magnitude. Dedicated kernels serve rate-buffer and collateral-conversion-reserve sizing using the same procedure.

ZK provability. The serving computation (kernel lookup, interpolation, and optimization) is deterministic given the kernel snapshot and borrower parameters. DALOC produces a zero-knowledge proof attesting that the quote was correctly derived from the declared kernel hash and risk policy. Settlement contracts verify this proof alongside the market’s state-transition fact, ensuring that not only the executed loan terms but the underwriting logic that produced them is cryptographically auditable. Kernel precomputation, which involves stochastic Monte Carlo simulation, sits outside the proof boundary; what is proven is the deterministic path from a committed kernel to a specific quote.

7.5 Funding and Execution Reserves

For liquidity-forwarding loans, the engine sizes separate reserves for funding mismatch and execution cost. The rate buffer covers the expected shortfall of the maximum pathwise funding deficit over the loan term; the collateral conversion reserve covers stressed round-trip execution costs. Both are calibrated jointly so that a collateral crash, higher utilization, thinner liquidity, and longer settlement delay can occur in the same path [4, 5, 7, 18].

External-venue liquidity stress is treated as a first-class scenario: simulations include high utilization, borrow-rate spikes, reserve pauses, delayed withdrawals, chain downtime, bridge delay, and failed or partial unwinds. A route is rejected when the required reserve, liquidation latency, or venue exposure exceeds the market policy or underwriter policy.

7.6 Pricing

Each quote is priced from expected gross credit loss, the incremental tail capital consumed by the first-loss buffer, expected hedging costs, and operating costs [5, 14, 26]. Each underwriter optimizes its own objective within constraints set by the committed precomputation kernel and its published policy; quote competition selects the best compatible terms among feasible offers.

7.7 Portfolio Aggregation and Concentration Risk

Each new loan is assessed against the existing book, not on a standalone basis. Portfolio aggregation is performed against the current precomputation kernel (Section 7.4), which provides the committed risk surface used for quote generation, reserve sizing, and admissibility checks. Concentration limits cover

counterparty, collateral asset, principal asset, external venue, bridge path, oracle source, infrastructure dependency, and market-operator dimensions.

The engine measures wrong-way risk: cases in which collateral falls while funding rates, liquidation costs, venue utilization, bridge latency, or the probability of venue failure rise. Diversification credit is recognized only when offsets remain legally, technically, and operationally available through the same close-out process [12, 29]. A portfolio that appears diversified by asset ticker may still be concentrated in the same oracle, bridge, liquidation venue, external lending protocol, or settlement operator; these shared dependencies are treated as concentration factors rather than ignored because the labels look different.

7.8 Operational Risk and Model Governance

Market and funding risks are estimated probabilistically where sufficient data exist. Sparse infrastructure failures (smart-contract, bridge, oracle, chain-halt, proof-system) are controlled through deterministic scenarios, exposure caps, and conservative add-ons rather than spuriously precise event frequencies. Each quote records the data snapshot, model hash, and calibration window used to produce it; recalibration is triggered when realized conditions deviate materially from model assumptions [3, 30].

8 Cross-Chain Exposure

Cross-chain operation is the default; same-chain loans are a special case. Two concerns arise at different layers.

Authorization (protocol-level): proving that collateral was pledged on the collateral chain so the principal chain's contracts can act on it. DALOC uses zero-knowledge storage proofs: cryptographic attestations that a particular storage slot held a particular value at a particular block, verified against the chain's state root. A state root relay or light client makes the source chain's finalized state available on the destination chain.

Token movement (proof-specified): bridge hooks in the creation proof move the borrowed asset to its destination. The operator selects the admissible bridge path when assembling the proof; keepers submit the resulting transaction on-chain for execution.

8.1 Authorization Flow

Consider a borrower who pledges WETH on Ethereum and requests USDC on Arbitrum. The collateral-chain issuer records the pledge; a settlement program reads the CDP's storage slot from Ethereum's state root via a storage proof (a Merkle inclusion proof against the state trie, attested by a finalized state root relayed to the destination chain), producing a verifiable claim that the collateral

is locked. On Arbitrum, the debt issuer validates this claim through the facts registry and issues the debt token.

Coordination relies on shared market identifiers and deterministic addressing, not real-time cross-chain messaging. Bridge latency affects capital delivery but not settlement correctness; a delayed bridge stalls execution, never produces an invalid state. The relay’s trust model is load-bearing for every cross-chain position: a compromised relay could feed false finality, enabling fabricated storage proofs. Participants inherit the relay’s security model alongside the oracle’s.

8.2 Deterministic Addressing

Identical hub addresses on every supported chain, combined with deterministic CREATE2 loan contracts, allow collateral on chain A and debt on chain B to reference the same loan identifier. Adding a new destination chain requires only a bridge adapter whitelisted on the hub.

8.3 Hyperliquid Margin Agent

For Prime Trade, the margin pipeline (Section 5) routes borrowed USDC to a per-loan *margin agent* on HyperEVM [16] at a CREATE2 address computed at quote time. Capital enters HyperCore as margin, never the borrower’s wallet. Permitted actions are restricted to perpetual trading; withdrawals, spot transfers, and external lending are blocked.

On close or liquidation, the margin agent recalls capital from HyperCore and bridges it back to Ethereum. Collateral never crosses to HyperEVM; only USDC round-trips. Figure 3 illustrates the full origination, active trading, and close cycle.

8.4 Effective Leverage

For Prime Trade, the borrowing LTV combined with venue leverage creates a multiplicative effect. For collateral worth C deposited at LTV λ with venue leverage ℓ , the notional controlled is

$$N = C \cdot \lambda \cdot \ell, \quad (8.1)$$

and the effective leverage relative to the collateral is $\lambda_{\text{eff}} = N/C = \lambda \cdot \ell$.

At $\lambda = 62.5\%$ LTV with $\ell = 5\times$ venue leverage, $\lambda_{\text{eff}} = \lambda \cdot \ell \approx 3.1\times$; this yields substantial notional controlled without selling the underlying. Collateral remains on Ethereum throughout.

9 Settlement and Verification

9.1 Event-Driven Settlement

Settlement is event-driven: each proven action executes atomically on submission, with no batch window and no global ordering. The detailed mechanics of proof submission, keeper economics, fee replacement, and cross-chain failure recovery are specified in the companion paper [20]; this section covers the settlement framework as it relates to the protocol’s product and risk architecture.

9.2 Settlement Programs

On-chain contracts are settlement executors: they verify proofs and execute committed transfers and hook chains but contain no market-specific logic. Market behavior (interest computation, liquidation detection, oracle selection, collateral policy, and compliance) is computed by zero-knowledge programs off-chain. Those programs output facts: hash-bound statements tying a program hash, market identifier, chain, prior state, new state, and any computed values to a verified proof.

The contracts do not recompute the market logic. Instead, they reconstruct the expected fact hash from their fixed parameters and the submitted leaf or state transition, check that the facts registry marks that fact as valid, and apply the corresponding transfer, accrual, liquidation, or update only if the fact is present. What is verified on-chain is therefore not an oracle feed or risk model directly, but the existence of a valid proof that the declared program produced the declared output from the declared inputs.

Programs split by economic cadence. **Matching** is one-time per loan: transfers, debt issuance, CDP authorization, and criteria-compatibility evaluation (Section 3). **Interest accrual** is per-action, submittable by any party for any debt at any time. **Health-check sweep** evaluates all active positions, flagging those that breach thresholds. **Per-position enforcement** proves a single position breaches its threshold, producible by either counterparty independently of the operator. **Close** is terminal after repayments and accrual catch up; either counterparty can produce the close proof.

All programs submit results through a single *facts registry* that accepts a fact hash and returns whether it is valid. Two on-chain fact structures carry all market state. A **state-transition fact** authorizes a state update, binding the settlement program, chain, issuer, and the previous and next roots (each encoding the full set of Merkle roots). An **interest fact** authorizes accrual on one loan’s debt, binding the rate program, chain, debt identifier, accrual window, and computed interest amount. The on-chain contract reconstructs the expected fact from its fixed parameters, checks the registry, and applies values only if valid. Matching, origination, liquidation-detection, close, and enforcement proofs produce state-transition facts that incorporate their respective logic; the two on-chain

structures are the common execution interface through which all program types take effect.

9.3 Commitment Registry

Each operator maintains an on-chain commitment registry scoped per market [20]. For every active position the registry stores a commitment hash alongside a storage-proved counter. The counter makes monitoring completeness enforceable: a health-check sweep proof must reference exactly as many positions as the storage-proved counter records at the observed block. Position terms (rates, thresholds, maturity dates, counterparty identities) remain off-chain as pre-images; only commitment hashes and the count appear on-chain. The companion paper analyzes the resulting privacy model and its limitations [20].

9.4 Market and Loan Contracts

Each market has two contracts. The **collateral manager issuer** (collateral chain) manages state, executes Merkle-authorized transfers, and deploys per-loan collateral managers. The **debt issuer** (principal chain) manages state and deploys per-loan issued debt tokens. Neither contains market-specific logic.

Each loan also has two contracts. The **collateral manager** (CDP) holds pledged assets and executes hook-composed pipelines; permitted operations are defined by the market's ZK programs. The **issued debt** is a per-loan ERC-20 tracking accrual via verified facts and partitioning repayment into principal reserve, interest reserve, and protocol fees (Section 10).

When position assets live on different chains, contracts deploy independently on each and storage proof finality lag applies. Cross-chain partial failures are detected and recovered via storage proofs [20].

Chain downtime is treated as a liveness failure, not a correctness failure. The affected issuer or adapter cannot execute hooks or process lifecycle events until block production resumes; the market resumes from its last valid state root. During the halt, economic risk can continue to move on other chains (oracle prices change, margin positions gain or lose value), so markets treat chain downtime as a modeled liveness risk in their admissibility checks, liquidation-latency bounds, and underwriter commitments.

9.5 Oracle Freedom and Permissionless Markets

Oracle logic is embedded in the market's zero-knowledge programs. Programs read price or rate data from on-chain contracts via storage proofs, enforcing freshness inside the proof rather than trusting off-chain feeds. Variable-rate positions carry a second oracle dependency: the external rate index referenced by the accrual circuit. Markets can be created from governance-registered program hashes without further per-market approval; registering wholly new program

types requires hub governance. The protocol verifies program outputs but does not audit program logic. Assessing a market’s risk properties is the participant’s responsibility.

9.6 System Invariants

Five product-layer invariants hold across all states. The companion paper [20] defines five safety invariants that overlap with and extend the list below, adding no unauthorized movement and no liquidation without detection; together they form the full security model:

1. **Token conservation.** For every asset, total balances across all market contracts equal deposited collateral plus supplied principal minus withdrawals. No operation creates or destroys tokens.
2. **No settlement without proof.** Every state transition requires a valid zero-knowledge proof registered in the facts registry. No state change can occur without cryptographic verification.
3. **No selective omission.** Health-check sweeps must cover all active positions; the storage-proved counter rejects incomplete sets.
4. **Position isolation.** Each loan’s contracts are independently deployed, independently liquidatable, and share no mutable state with any other loan. One position’s insolvency cannot reach into another’s contract state. (Economic correlation through a shared underwriter may affect NAV but does not violate contract-level isolation.)
5. **Exit without cooperation.** Either counterparty can produce per-position close, enforcement, and accrual proofs independently. When a position is healthy and not yet matured but the operator has been absent past the escape timeout, any pre-image holder can settle at oracle price.

10 Position Tokens

Each funded loan produces a tokenized position that represents the lender’s claim, carries accrual, and enforces solvency rules as a transferable ERC-20.

10.1 Debt Tokens

Each funded loan issues a transferable ERC-20 whose face value is one unit of the principal asset per token. The token is the lender’s economic position. The claim value per token v (the amount redeemable at maturity) is

$$v = 1 + I_{\text{cum}}, \quad (10.1)$$

where I_{cum} is the cumulative interest accrued per token, rising as interest is verified. Under fixed-rate terms, interest accrues linearly (simple interest, not compound), so the borrower’s total obligation at maturity for principal P over

term T is

$$D = P \left(1 + r \frac{T}{T_{\text{year}}} \right), \quad (10.2)$$

and the per-period increment used by the on-chain accrual program is

$$\Delta I = P r \frac{\Delta t}{T_{\text{year}}}, \quad (10.3)$$

where P is the outstanding principal in token units, r the annualized rate, Δt the accrual period in seconds, and $T_{\text{year}} = 365.25 \times 86,400$. Linear accrual is a deliberate choice: the borrower's obligation is fully determined at origination, with no path-dependent compounding.

Variable-rate positions reference an external on-chain rate index; the accrual circuit reads the index value via storage proof, making the rate determination verifiable. The per-period increment follows the same formula with r replaced by the proven index rate for that interval; the index oracle is a market parameter and constitutes a second oracle dependency beyond the collateral price oracle.

Each accrual requires a valid interest fact verified through the facts registry (Section 9). On-chain guards add defense in depth: accrual windows advance monotonically, no accrual extends past the close timestamp, and cumulative amounts cannot exceed what the committed rate permits. These bounds hold independently of the proof system [20].

Repayment proceeds enter a principal reserve. A subsequent accrual step, backed by a proof tied to the market's rate program, splits the interest portion into a protocol fee and an interest reserve. Holders redeem by burning tokens and receiving principal plus their share of the interest index.

A strict accounting identity holds at all times, ensuring no principal, interest, or fee is created or destroyed:

$$R_{\text{principal}} + R_{\text{interest}} + F_{\text{protocol}} = B_{\text{token}}, \quad (10.4)$$

where $R_{\text{principal}}$ is the redeemable principal reserve, R_{interest} the interest reserve, F_{protocol} the accrued protocol fees, and B_{token} the contract's token balance. Redemptions are blocked until interest for the latest period is accrued, preventing exits at a stale index. A loan can be marked closed only when accrual is current and the principal reserve covers all outstanding tokens.

Debt tokens from criteria-restricted markets carry transfer-time criteria (e.g. "KYC tier ≥ 2 ") enforced at the hub-routed transfer step, ensuring restrictions persist after origination. When an underwriter is creditor, underwriter shares (Section 6) reflect the aggregate claim value across all held debt tokens. Debt tokens are ERC-20 with transfer routing through the hub; this hub routing enables criteria enforcement while preserving standard ERC-20 composability for unrestricted tokens.

11 Governance and Trust Assumptions

The protocol's trust surface is deliberately narrow. Market parameters are fixed at creation and identified by their hash; every settlement action requires a valid zero-knowledge proof registered in the facts registry (Section 9). Trust concentrates in the hub contract, which governs all capital flows, and in the liveness of off-chain infrastructure.

Beyond these trust surfaces, the protocol minimizes dependence on any single party. Markets are independent state machines (Section 3) that maintain their own state without shared global state; market creation from registered program hashes is permissionless (new program types are governance-gated), and so is participation as an operator, liquidator, or keeper. Intent cancellation, margin adjustment, and direct repayment are on-chain user actions requiring no operator cooperation; per-position enforcement proofs and escape settlement provide exit guarantees without operator participation [20].

Safety is structural: the system invariants of Section 9 hold across all reachable states, and no state transition can occur without a valid zero-knowledge proof. Policy compliance is enforced at proof time rather than through execution-blocking (Section 6). Liveness depends on each market's operator and on the chains, bridges, provers, and external venues used by that market; a configurable timeout bounds how long a market can stall before the operator is paused, per-position exit paths remain available, and after a longer escape timeout any pre-image holder can settle healthy, non-matured positions at oracle price. The decentralization properties, including correctness, completeness, and exit guarantees, are analyzed in the companion paper [20].

The architecture targets custody recoverability without reliance on the original off-chain operator. Users can submit cancellations, allowance revocations, share transfers, and recovery actions directly to the relevant on-chain contracts; for markets deployed on L2s, this may rely on that L2's L1 inbox, escape hatch, or forced-transaction mechanism. This does not mean that every economic action is immediately available from L1: liquidation, oracle updates, bridge movement, and external-venue unwinds still depend on the relevant chains and infrastructure being live. Each market defines its own data-availability and recovery policy, and the architecture admits progressive trust reduction as markets mature.

11.1 Protocol Administration

A multisig with timelock governs protocol contracts, which are upgradeable. Upgradeability permits response to critical vulnerabilities at the cost of trust in the multisig. Because core contracts control capital flows and settlement execution, an upgrade can alter routing and pause behavior across every market simultaneously. Pause and bridge-adapter whitelisting are also multisig actions; cross-chain routes are permissioned even though markets are not.

Proof programs (circuits and verification keys) and underwriter policies are immutable once registered; the upgrade mechanism cannot alter what conditions must be proven, only what effects the contract executes after a valid proof is verified. The residual trust implications of hub upgradeability are analyzed in the companion paper [20].

11.2 Liveness Dependencies

Settlement correctness is enforced by zero-knowledge proofs (Section 9), but liveness is not: if a market's off-chain infrastructure halts, no new settlement actions execute for that market. Existing positions remain safe (collateral stays locked, debt is unchanged, intent cancellation remains available), but under-collateralized positions go undetected by operator sweeps until the operator resumes. Either counterparty can independently detect and flag breached positions via per-position enforcement proofs at any time. To enable this, operators publish position pre-images to persistent off-chain storage at origination; the publication layer falls back from the operator's API to on-chain calldata to decentralized storage, ensuring counterparties retain the data needed for independent enforcement [20].

Each market defines a liveness timeout. Sweep proving scales linearly with portfolio size; a large book requires proportionally more proving time. If no operator sweep completes within that window, the operator is paused: no new originations until a sweep is delivered. Per-position operations by counterparties continue unaffected. The three-level exit hierarchy (per-position independence, operator pause, and escape timeout) is specified in the companion paper [20].

LTV limits, rates, and buffer sizes are produced off-chain by the risk engine. The serving computation is cryptographically verified via zero-knowledge proof (Section 7), ensuring quotes derive correctly from the declared kernel and risk policy. The underwriter's committed policy (Section 6) imposes outer bounds (rate floors, LTV ceilings, concentration caps) enforced at proof time. Neither mechanism validates the underlying model calibration or Monte Carlo assumptions between those bounds.

Cross-chain movement depends on the selected bridge infrastructure and state root relay (Section 8); the protocol inherits the bridge's and relay's liveness and censorship properties. Authorization correctness is cryptographic (ZK storage proofs over actual chain state), but proof-generation availability depends on the data provider, and bridge execution depends on the source and destination chains being live.

12 Discussion

12.1 Risks and Limitations

Smart-contract risk. The hub, issuers, CDPs, and debt tokens share common contract logic. A critical vulnerability in a widely-used contract could affect multiple markets. Formal verification covers token conservation; broader functional coverage is an ongoing effort.

Oracle staleness. Liquidation detection depends on timely oracle updates. If oracle prices lag during a flash crash, detection is delayed and recovery proceeds may fall below debt. The flag-gated model bounds this by allowing any party to produce a per-position enforcement proof the moment prices update, but oracle staleness during the gap window remains a residual risk.

Proof-system failures. If the ZK prover becomes unavailable, no new settlement actions can execute. Existing positions are safe (funds remain in CDPs) but cannot settle new interest or originate. Recovery requires a new prover deployment. No fallback to optimistic settlement is currently implemented.

Liquidation race with external venues. A liquidity-forwarding loan exposes the underwriter to the gap between DALOC's flag-gated liquidation and the venue's instant liquidation trigger. Reserve sizing (Section 7) attempts to make this gap negligible, but a sufficiently sharp collateral crash can trigger the venue's liquidation before DALOC's detection completes, leaving a residual shortfall absorbed by the first-loss buffer.

Cross-chain liveness. Bridge delays or outages stall capital delivery on Prime Trade origination and close. Positions remain valid but economically frozen until the bridge resumes. Markets that rely on a single bridge path inherit its liveness profile.

Host-chain downtime. DALOC markets inherit the liveness of the chains on which their contracts are deployed. A chain halt delays settlement execution and liquidations while economic risk continues to move elsewhere. Markets on L2s additionally inherit the rollup's recovery properties. Chain downtime is a modeled liveness risk (Section 9), not a correctness failure.

External-venue liquidity and liveness. Liquidity-forwarding loans depend on external venues for borrowing, repayment, and liquidation timing. If a venue pauses, has insufficient liquidity, or liquidates faster than DALOC's detection process, the loan may enter a pending-unwind state and residual losses flow through the loss waterfall (Section 6). Reserve sizing makes this risk explicit and priced but cannot remove it.

Prime Trade venue downtime. Prime Trade positions depend on HyperEVM and HyperCore remaining live. If the destination venue stalls, the margin agent cannot rebalance, stop out, or recall capital on schedule. The position remains

constrained by its program, but economic exposure changes while execution is unavailable. Venue downtime is modeled in the underwriter's policy.

Program correctness. Settlement proofs verify that a declared program produced the declared output from the declared inputs; they do not verify that the program itself is correct. A bug in a market's zero-knowledge program (accrual, liquidation, oracle) can cause systematic mispricing or missed liquidations for every position in that market. Formal verification of program logic is ongoing.

Variable-rate index oracle. Variable-rate positions introduce a second oracle surface beyond collateral price. If the referenced rate index is stale, manipulated, or unavailable, accrual computes over an incorrect rate. This risk is bounded by the market's freshness constraints and the underwriter's committed policy, but not eliminated [20].

State root relay. Cross-chain operations depend on a state root relay or light client whose trust model is load-bearing: if the relay is compromised, storage proofs can be forged against fabricated state. The relay is not named as a specific product; its concrete trust model (operator set, finality delay, data availability) is a per-deployment residual trust assumption [20].

Operator information timing. The operator observes sweep results before on-chain submission, creating an asymmetric information window exploitable through external markets. Per-position enforcement (producible by either counterparty) bounds the window but does not eliminate it [20].

Stablecoin and bridge centralization. For USDC-based markets, the principal asset issuer (Circle) can freeze balances at specific addresses. If a CDP, underwriter, or margin agent address is blacklisted, the affected positions cannot settle normally. This concentration risk is mitigated by market diversity (alternative principal assets and bridge adapters may serve separate markets) but cannot be eliminated while a single issuer's token is the primary path.

12.2 Future Capabilities

The architecture is designed for staged extension. Near-term additions include:

- **Under-collateralized credit.** On-chain reputation, verifiable off-chain creditworthiness (e.g. institutional attestations), and cross-protocol track records enable LTV thresholds above 100% for qualified borrowers under tighter underwriter oversight.
- **Cross-margining.** Correlated positions under the same borrower and same underwriter can share margin via a portfolio-level health factor, improving capital efficiency without weakening per-position isolation at the settlement layer.
- **Options and nonlinear collateral.** Collateral-class adapters for options and other nonlinear claims, valued at stressed executable close-out value rather than oracle mark, extend the protocol beyond spot and yield-bearing tokens.

- **Multi-venue Prime Trade.** Margin agents generalize beyond HyperEVM to additional perpetual venues, each as a separate bridge adapter and constraint set.
- **Underwriter ecosystem.** The architecture supports multiple independent underwriters, each with a distinct policy (one conservative with high-grade collateral and short tenors, another aggressive with broader collateral and higher concentration limits). Competition between underwriters for LP capital creates pressure toward better risk-adjusted returns.
- **Institutional access.** Criteria-based matching (Section 3) already enables regulated counterparties to participate without a permissioned fork. Realizing this in production requires only attestation issuers and criteria definitions, not protocol modification.
- **Maturity management.** Refinancing (Section 5) already allows mid-term rollovers. At scale, automated maturity laddering (where the operator pre-matches refinancing quotes ahead of expiry) can smooth redemption waves, reduce cliff-event risk, and improve capital utilization across the loan book.

12.3 Conclusion

DALOC unifies collateralized credit, cross-chain capital coordination, and risk quantification within a single settlement layer. Underwriters fund loans directly under a proof-attested policy; external venues provide overflow capacity beyond the underwriter’s balance sheet. Fixed- and variable-rate, fixed-term loans; leveraged perpetual trading without selling the underlying; and passive yield from a portfolio of loans share a common matching and settlement infrastructure.

Criteria-based matching lets restricted and unrestricted counterparties share a single liquidity surface. Zero-knowledge proofs enforce settlement correctness and underwriting integrity; policy compliance is enforced at proof time. Proposition and per-market isolation bound the blast radius of defaults. The protocol presents an architecture for the transition from pool-based, variable-rate, single-chain lending to structured, provable, multi-chain credit.

References

- [1] Aave. Aave V4 overview. Technical report, Aave, 2026. URL <https://github.com/aave/aave-v4/blob/main/docs/overview.md>.
- [2] Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999. doi: 10.1111/1467-9965.00068.
- [3] Massimo Bartoletti and Enrico Lipparini. A theory of lending protocols in DeFi, 2025. URL <https://arxiv.org/abs/2506.15295>.
- [4] Mahsa Bastankhah, Viraj Nadkarni, Xuechao Wang, and Pramod Viswanath. AgileRate: Bringing adaptivity and robustness to DeFi lending markets, 2024. URL <https://arxiv.org/abs/2410.13105>.
- [5] Bastien Baude, Damien Challet, and Ioane Muni Toke. Optimal risk-aware interest rates for decentralized lending protocols, 2025. URL <https://arxiv.org/abs/2502.19862>.
- [6] Carlos Castro-Iragorri, Julian Ramirez, and Sebastian Velez. Financial intermediation and risk in decentralized lending protocols, 2021. URL <https://arxiv.org/abs/2107.14678>.
- [7] Tarun Chitra, Peteris Erins, and Kshitij Kulkarni. Attacks on dynamic DeFi interest rate curves, 2023. URL <https://arxiv.org/abs/2307.13139>.
- [8] Jonathan Chiu, Emre Ozdenoren, Kathy Yuan, and Shengxing Zhang. On the fragility of DeFi lending. Staff Working Paper 2023-14, Bank of Canada, 2023. URL <https://www.bankofcanada.ca/2023/02/staff-working-paper-2023-14/>.
- [9] DefiLlama. DefiLlama — DeFi dashboard. <https://defillama.com>, 2026.
- [10] Euler Labs. Euler v2 whitepaper. Technical report, Euler Labs, 2024. URL <https://docs.euler.finance/euler-v2/about/whitepaper>.
- [11] Mathis Gontier Delaunay, Paul Frambot, Quentin Garchery, and Matthieu Lesbre. Morpho blue whitepaper. Technical report, Morpho Labs, 2023. URL <https://github.com/morpho-org/morpho-blue/blob/main/morpho-blue-whitepaper.pdf>.
- [12] Michael B. Gordy. A risk-factor model foundation for ratings-based bank capital rules. *Journal of Financial Intermediation*, 12(3):199–232, 2003. doi: 10.1016/S1042-9573(03)00040-8.
- [13] Lewis Gudgeon, Sam M. Werner, Daniel Perez, and William J. Knottenbelt. DeFi protocols for loanable funds: Interest rates, liquidity and market efficiency, 2020. URL <https://arxiv.org/abs/2006.13922>.

- [14] Joseph Y. Halpern, Rafael Pass, and Aditya Saraf. Fair interest rates are impossible for lending pools: Results from options pricing, 2024. URL <https://arxiv.org/abs/2410.11053>.
- [15] Lioba Heimbach, Eric G. Schertenleib, and Roger Wattenhofer. Short squeeze in DeFi lending market: Decentralization in jeopardy?, 2023. URL <https://arxiv.org/abs/2302.04068>.
- [16] Hyperliquid Labs. Hyperliquid: A performant L1 for a decentralized financial system. <https://hyperliquid.xyz>, 2024.
- [17] Robert C. Merton. On the pricing of corporate debt: The risk structure of interest rates. *The Journal of Finance*, 29(2):449–470, 1974. doi: 10.1111/j.1540-6261.1974.tb03058.x.
- [18] Viraj Nadkarni and Pramod Viswanath. Split the yield, share the risk: Pricing, hedging and fixed rates in DeFi, 2025. URL <https://arxiv.org/abs/2505.22784>.
- [19] Bhargav Nagaraja Bhatt, Paul Frambot, Quentin Garchery, Mathis Gontier Delaunay, Adrien Husson, Paul-Adrien Nicole, Adrien Laversanne-Finot, and Matthieu Lesbre. Morpho midnight whitepaper. Technical report, Morpho Labs, 2025. URL <https://github.com/morpho-org/morpho-midnight/blob/main/morpho-midnight-whitepaper.pdf>.
- [20] Bartosz Nowak, Marcello Bardus, and Maciej Sulecki. DALOC: Decentralization architecture for credit coordination. Technical report, Herodotus, 2026.
- [21] Daniel Perez, Sam M. Werner, Jiahua Xu, and Benjamin Livshits. Liquidations: DeFi on a knife-edge. In *Financial Cryptography and Data Security*, pages 457–476. Springer, 2021. doi: 10.1007/978-3-662-64331-0_24. URL <https://arxiv.org/abs/2009.13235>.
- [22] Kaihua Qin, Liyi Zhou, Pablo Gamito, Philipp Jovanovic, and Arthur Gervais. An empirical study of DeFi liquidations: Incentives, risks, and instabilities. In *Proceedings of the 21st ACM Internet Measurement Conference*, pages 336–350. ACM, 2021. doi: 10.1145/3487552.3487811. URL <https://arxiv.org/abs/2106.06389>.
- [23] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3):21–41, 2000. doi: 10.21314/JOR.2000.038.
- [24] Agathe Sadeghi and Zachary Feinstein. Liquidation dynamics in DeFi and the role of transaction fees, 2026. URL <https://arxiv.org/abs/2602.12104>.

- [25] Sky (formerly MakerDAO). SparkLend. Technical report, Sky, 2023. URL <https://docs.spark.fi>.
- [26] Lukasz Szpruch, Marc Sabaté Vidales, Tanut Treetanthiploet, and Yufei Zhang. Pricing and hedging of decentralised lending contracts, 2024. URL <https://arxiv.org/abs/2409.04233>.
- [27] Phoebe Tian and Yu Zhu. Liquidation mechanisms and price impacts in DeFi. Staff Working Paper 2025-12, Bank of Canada, 2025. URL <https://www.bankofcanada.ca/2025/03/staff-working-paper-2025-12/>.
- [28] Natkamon Tovanich, Myriam Kassoul, Simon Weidenholzer, and Julien Prat. Contagion in decentralized lending protocols: A case study of compound. In *Proceedings of the 2023 Workshop on Decentralized Finance and Security*. ACM, 2023. doi: 10.1145/3605768.3623544. URL <https://hal.science/hal-04221228>.
- [29] Anastasiia Zbandut and Carolina Goldstein. Institutionalizing risk curation in decentralized credit, 2025. URL <https://arxiv.org/abs/2512.11976>.
- [30] Anastasiia Zbandut and Carolina Goldstein. Vault as a credit instrument, 2026. URL <https://arxiv.org/abs/2604.17579>.